

Manual Tecnico

Manual Tecnico

1. Resumen

QHERE es una aplicacion web para control de asistencia escolar con arquitectura separada en frontend React, backend Flask y plataforma Supabase para autenticacion, base de datos y almacenamiento de evidencias.

2. Arquitectura

- Capa de presentacion: `frontend/src/pages`
- Capa de componentes: `frontend/src/components`
- Capa de integracion: `frontend/src/lib` y `frontend/src/api`
- Capa backend: `backend/app/routes`
- Capa de datos: Supabase PostgreSQL + Storage

3. Modelo de datos principal

- `schools`: configuracion del centro educativo
- `profiles`: usuarios y roles
- `grade\_sections`: cursos, secciones y turnos
- `schedules`: horarios oficiales
- `school\_calendar`: feriados, vacaciones y eventos
- `students`: estudiantes y QR
- `parents`: relacion padre-estudiante
- `student\_teachers`: asignacion academica
- `attendance`: entrada, salida, estado y tardanza
- `excuses`: justificaciones con evidencia
- `notifications`: notificaciones internas
- `audit\_log`: trazabilidad operativa
- `authorized\_devices`: control de dispositivos
- `attendance\_geo\_events`: geolocalizacion opcional
- `notification\_queue`: cola de alertas
- `gradebook\_entries`: base para integracion academica

4. Flujo tecnico

1. El usuario autentica contra Supabase Auth.
2. El frontend consulta `profiles` para resolver el rol y el acceso.
3. El admin configura escuela, secciones, horarios y calendario.
4. El docente registra asistencia por QR o manual usando Flask.
5. El backend valida permisos, QR, duplicidad, dispositivo y geolocalizacion.
6. El backend escribe asistencia y auditoria en Supabase.
7. El padre consulta historial y envia excusas con evidencia al bucket `excuses`.

5. Backend Flask

Rutas principales:

- `auth\_routes.py`: autenticacion y perfil
- `attendance\_routes.py`: escaneo, salida, manual, alertas y auditoria
- `student\_routes.py`: CRUD base de estudiantes
- `teacher\_routes.py`: consulta de docentes y asistencia
- `excuse\_routes.py`: flujo de excusas

6. Seguridad

- Roles por perfil: `admin`, `teacher`, `parent`
- Validacion de centro y aprobacion de perfil
- Dispositivos autorizados para asistencia
- Bucket exclusivo para evidencias
- Bitacora de acciones en `audit\_log`

7. Instalacion tecnica

Frontend

```
cd frontend  
npm install  
npm run dev
```

Backend

```
cd backend  
python -m venv venv  
venv\Scripts\activate  
pip install -r requirements.txt  
python run.py
```

Base de datos

1. Ejecutar `database/schema.sql`
2. Ejecutar migraciones de `database/migrations/`
3. Ejecutar `database/future\_tables.sql` si se desea habilitar funcionalidades opcionales avanzadas

8. Observaciones de despliegue

- El frontend compila con `npm run build`
- El backend depende de variables de entorno de Supabase
- Las alertas por correo pueden procesarse con `python process\_notification\_queue.py`
- El envio SMTP requiere `SMTP\_HOST`, `SMTP\_PORT`, `SMTP\_USERNAME`, `SMTP\_PASSWORD` y `SMTP\_FROM\_EMAIL`